



ZUCCHETTI



AI TrAIning

Specifica tecnica

Filippo Sabbadin

08 / 08 / 2025

Indice

1. Introduzione	1
1.1. Scopo del documento	1
1.2. Scopo del prodotto	1
1.3. Glossario	2
1.4. Riferimenti	2
1.4.1. Riferimenti normativi	2
1.4.2. Riferimenti informativi	2
1.4.3. Riferimenti tecnici	3
2. Tecnologie	4
3. Premessa	5
3.1. Nodi	5
3.2. Scene	5
3.3. Segnali	6
4. Architettura	8
4.1. Introduzione	8
4.1.1. Funzioni comuni	8
4.2. Asset comuni	9
4.2.1. Giocatore	9
4.2.1.1. Movement	10
4.2.1.2. CameraRayCast	10
4.2.1.3. StateMachine	12
4.2.1.4. PlayerUI	14
4.2.1.5. PlayerSavesHandler	15
4.2.1.6. GrabItem	15
4.2.1.7. ParticleEmitter	15
4.2.2. Interazione	16
4.2.2.1. InteractableArea	16
4.2.2.2. Control	16
4.2.2.3. NPC	17
4.2.2.4. InteractableSign	18
4.2.2.5. NPCDialogue	18
4.2.3. Dialoghi	18
4.2.3.1. Dialogue	19
4.2.3.2. DialogueBoxSimple	19
4.2.3.3. DialogueBoxOptions	19
4.2.3.4. DialogueOptionsButtons	19
4.2.4. Salvataggi	19

4.2.4.1. Saves	20
4.2.4.2. SavesHandler	20
4.2.5. Singletons / Autoloads	20
4.2.5.1. LevelsTransition	20
4.2.5.2. OptionsSave	20
4.2.5.3. InputUIHandler	20
4.3. Struttura base livello	21
4.3.1. PlayerSpawn	21
4.3.2. PauseMenu	21
4.4. Livello «Regressione lineare»	22
4.4.1. Cannone e grafico LR	22
4.4.1.1. LRCannon	23
4.4.1.2. LinearRegressionGraph	23
4.5. Livello «Albero di decisione»	23
4.5.1. Albero	23
4.5.1.1. DecisionNode	25
4.5.1.2. DecisionTree	25
4.5.1.3. CheckUnlocked	25
4.5.1.4. CollectibleSpawner	25
4.6. Livello «Causalità»	25
4.6.1. CausalitySavesHandler	26
4.6.2. ACUnit	26
4.6.3. ACUnits	26
4.6.4. CutscenesHandler	26
4.6.5. Scena di intermezzo	26
5. Requisiti soddisfatti	27
5.1. Tabella requisiti soddisfatti	27
5.2. Grafico requisiti soddisfatti	31

Lista di immagini

Figura 1	Scena del giocatore	6
Figura 2	Diagramma delle classi del giocatore	9
Figura 3	Diagramma delle classi della telecamera del giocatore	10
Figura 4	Diagramma sulla struttura della macchina di stati	12
Figura 5	Diagramma degli stati	12
Figura 6	Diagramma delle classi della UI del giocatore	14
Figura 7	Diagramma degli oggetti con cui il giocatore può interagire	16
Figura 8	Diagramma dei vari tipi di UI	16
Figura 9	Diagramma sul funzionamento di un dialogo	18
Figura 10	Diagramma sul funzionamento dei salvataggi	19
Figura 11	Classi <i>Autoloads</i>	20
Figura 12	Diagramma di un livello base	21
Figura 13	Diagramma sul funzionamento di un grafico «Linear Regression» nel gioco	22
Figura 14	Diagramma sul funzionamento dell'Albero di decisione	23
Figura 15	Diagramma del livello della causalità	25

Lista di tabelle

Tabella 1 Tecnologie utilizzate	4
Tabella 2 Requisiti di funzionalità	27

1. Introduzione

1.1. Scopo del documento

Lo scopo di questo documento è descrivere l'architettura e le scelte relative a essa che sono state fatte durante la fase di progettazione e codifica del progetto.

Vengono riportati i grafici UML delle classi per rappresentare l'architettura finale dell'applicazione.

1.2. Scopo del prodotto

Il progetto consiste nello sviluppo di un videogioco con il motore di gioco [Godot*](#) di tipo [platformer*](#), in cui il giocatore controlla un personaggio che si muove in un ambiente tridimensionale. Il gioco si basa su temi di [Intelligenza Artificiale*](#) e [Machine Learning*](#), sviluppando meccaniche ed elementi del livello basati su questi argomenti.

Comprende 3 livelli, ognuno con un tema diverso, un menu principale che viene visualizzato non appena il gioco viene avviato, e un menu di pausa che può essere visualizzato in qualsiasi momento durante il gioco.

Ogni livello presenta meccaniche diverse e uniche.

- **Regressione lineare**

Il livello presenta una serie di grafici con una linea e dei dati, accedendo ad un «cannone» il giocatore può posizionare dei nuovi dati nel grafico e la retta si modifica in base alla posizione del nuovo punto. Nel caso i punti siano stati piazzati male e non sia possibile modificare ulteriormente la linea, il giocatore può resettare il grafico premendo il tasto apposito.

Se soddisfatto della rotazione e posizione della linea il giocatore può uscire dal cannone e camminare sopra la linea per proseguire nel livello.

- **Albero di decisione**

In questo livello sono presenti diversi cani, ognuno di una razza diversa, ed un albero di decisione tridimensionale. Ad ogni nodo dell'albero viene mostrata una domanda e la direzione da seguire per ogni risposta.

L'obiettivo del giocatore è rispondere correttamente a ognuna di queste domande e posizionare il cane nel nodo finale giusto. In caso corretto, il giocatore potrà visualizzare la razza indovinata in un tabellone, e questo darà un Training Data al giocatore per ogni razza indovinata.

Infine è presente un NPC che permette al giocatore di riportare tutti i cani nella posizione iniziale nel caso siano troppo sparsi.

- **Causalità**

Appena caricato nel livello, il giocatore avrà la possibilità di parlare con un NPC gelataio che lo guiderà all'obiettivo di questo livello, cioè accendere tutte le unità di codizionatori esterne presenti, poiché crede che le vendite di gelati e l'uso dei condizionatori siano correlate. Nel livello sono presenti anche dei grafici che cambiano durante il livello in base alle azioni del giocatore.

Una volta accese tutte le unità, verrà avviata una [scena di intermezzo*](#) e alla fine di essa il giocatore dovrà indicare la vera causa della vendita di gelati.

1.3. Glossario

Per facilitare la comprensione del documento, è stato creato un glossario che contiene i termini utilizzati nel documento e le loro definizioni. I termini presenti nel glossario sono colorati di blu e seguiti da un'asterisco: [esempio*](#).

Il glossario è accessibile tramite il link:

<https://github.com/fliposab/ProgettoStage/blob/main/Documentazione/Glossario.pdf>

oppure consultando il rispettivo documento all'interno della stessa cartella.

1.4. Riferimenti

1.4.1. Riferimenti normativi

- Piano di lavoro:

<https://github.com/fliposab/ProgettoStage/blob/main/Documentazione/Piano-di-lavoro.pdf>

- Norme di progetto:

<https://github.com/fliposab/ProgettoStage/blob/main/Documentazione/Norme-di-progetto.pdf>

1.4.2. Riferimenti informativi

- Slide T05 del corso di Ingegneria del Software:

<https://www.math.unipd.it/~tullio/IS-1/2024/Dispense/T05.pdf>

- Diagrammi UML:\

- Documentazione «Godot Engine»:

<https://docs.godotengine.org/en/stable/>

1.4.3. Riferimenti tecnici

- Documentazione di Godot:

<https://docs.godotengine.org/it/4.x/about/introduction.html>

- I concetti chiave di Godot:

https://docs.godotengine.org/it/4.x/getting_started/introduction/key_concepts_overview.html

- La filosofia progettuale di Godot:

https://docs.godotengine.org/it/4.x/getting_started/introduction/godot_design_philosophy.html

- Architettura di Godot:

https://docs.godotengine.org/en/stable/contributing/development/core_and_modules/godot_architecture_diagram.html

2. Tecnologie

Nome	Descrizione	Versione
Codice		
GDScript	Linguaggio di programmazione di alto livello, con sintassi simile a Python, viene integrato con il motore di gioco Godot	(Legata a Godot)
GDSshader	Linguaggio simile a GLSL ES 3.0 usato per la creazione di materiali più complessi	(Legata a Godot)
Typst	Linguaggio utilizzato per la stesura dei documenti	0.13.1
Softwares		
Godot	Il motore di gioco open source per lo sviluppo del videogioco.	4.5-beta3-mono
Blender	Software di modellazione ed animazione 3D usato per creare i modelli 3D del gioco	4.4.3
Strumenti e servizi		
Git		2.50.1
GitHub Actions	Servizio di integrazione continua e distribuzione continua (CI/CD), utilizzato per compilare i documenti ad ogni push	-
Tipi di files non generati dagli strumenti elencati sopra		
*.csv	«Comma separated values», file utilizzato per memorizzare le frasi nelle lingue diverse supportate dal gioco	-
*.ini	Tipo di file «plain-text» utilizzato per salvare i dati del gioco	-
*.glb	«GLTF Binary», file utilizzato per memorizzare i modelli 3D e le loro animazioni in formato binario, in modo da risparmiare spazio e migliorare le prestazioni	2.0.1

Tabella 1: Tecnologie utilizzate

3. Premessa

Prima di iniziare a descrivere l'architettura del progetto, è necessario introdurre alcuni concetti chiave di Godot.

3.1. Nodi

Dalla documentazione di Godot:

I nodi sono i blocchi fondamentali del gioco. Sono come ingredienti in una ricetta. Ci sono dozzine di tipi che possono mostrare un'immagine, riprodurre un suono, rappresentare una camera, e molto altro. Tutti i nodi hanno le seguenti caratteristiche:

- *Un nome.*
- *Proprietà modificabili.*
- *Ricevono callback per aggiornarsi ad ogni frame.*
- *Si possono estendere con nuove proprietà e funzioni.*
- *Si possono aggiungere a un altro nodo come figlio.*

Inoltre ad ogni nodo si può assegnare uno script, che estende il tipo di quel nodo e aggiunge nuove funzionalità.

I principali tipi di nodi che vengono utilizzati in questo progetto sono:

- **Node**: nodo base da cui vengono estesi tutti gli altri nodi, in questo progetto viene usato per assegnare classi e inserirli come figli in altri nodi.
- **Node3D**: rappresenta un oggetto nello spazio tridimensionale.
 - **CharacterBody3D**: rappresenta un personaggio nel gioco, gestendo la sua posizione, animazione e interazioni.
 - **Camera3D**: rappresenta una telecamera nello spazio tridimensionale, che può essere utilizzata per visualizzare la scena.
 - **MeshInstance3D**: rappresenta un oggetto tridimensionale con una mesh, che può essere utilizzato per visualizzare modelli 3D.
 - **CollisionShape3D**: rappresenta una forma di collisione nello spazio tridimensionale, utilizzata per gestire le interazioni fisiche tra gli oggetti.
 - **Area3D**: rappresenta un'area nello spazio tridimensionale, utilizzata per gestire le interazioni tra gli oggetti all'interno di essa.
- **AnimationPlayer**: gestisce le animazioni degli oggetti nella scena, permettendo di riprodurre animazioni su mesh, telecamere e altri nodi.
- **Control**: rappresenta un nodo di interfaccia utente, utilizzato per gestire gli elementi dell'interfaccia grafica del gioco.

3.2. Scene

Dalla documentazione di Godot:

Quando organizzi nodi in un albero, come il nostro personaggio, possiamo chiamare questa

formazione una scena. Una volta salvata, la scena si presenta come un nuovo nodo nell'editor, dove possiamo aggiungerlo come figlio di un nodo esistente. In questo caso, l'istanza della scena appare come nodo singolo con interni nascosti. Le scene ci consentono di strutturare il codice del gioco in qualunque modo tu voglia. Puoi comporre nodi per creare nodi personalizzati e complessi, come un personaggio di gioco che si muove e salta, una barra della vita, una cesta con cui puoi interagire, e molto altro.

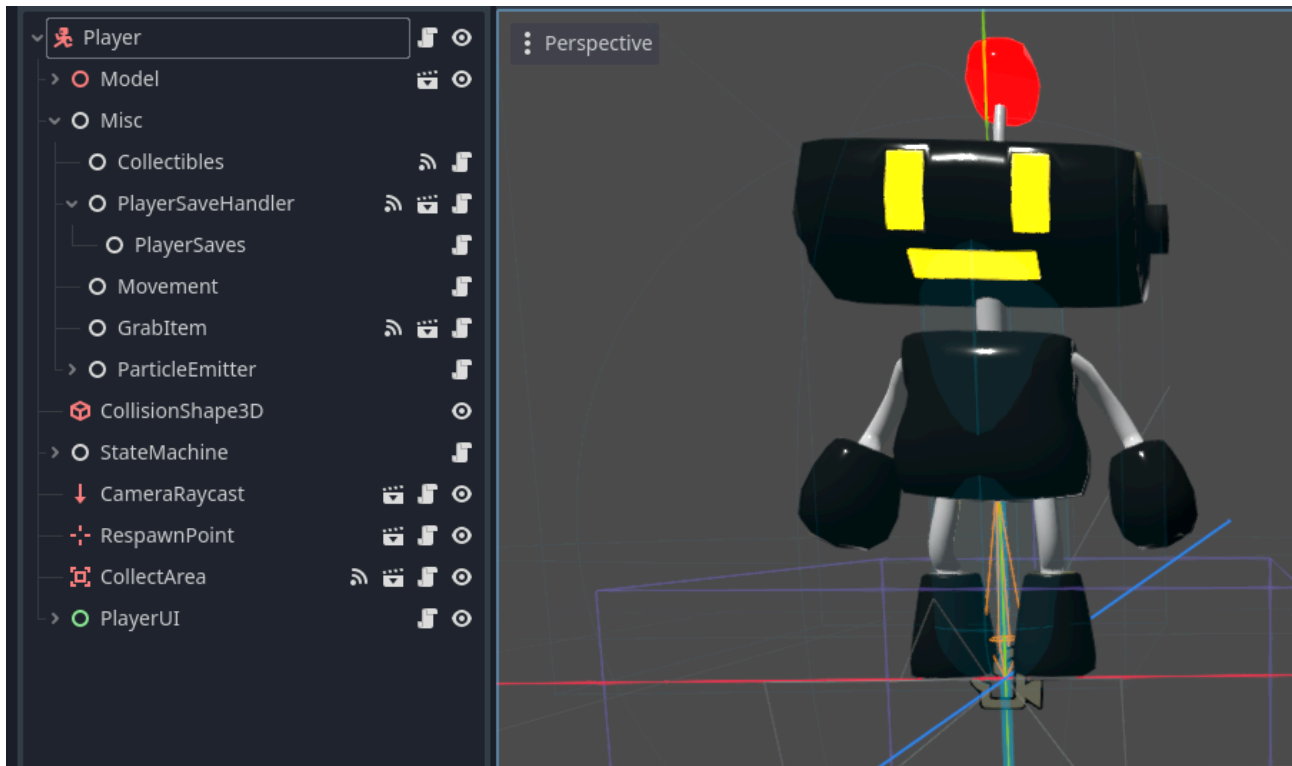


Figura 1: Scena del giocatore

Oltre che a comportarsi come nodi, le scene hanno anche le seguenti caratteristiche:

- Hanno sempre un nodo *owner* come il «Player» nel nostro esempio.
- Si possono salvare sul disco locale e caricarle in seguito.
- Si possono creare quante più istanze di una scena si desidera. Ad esempio, si possono avere cinque o dieci personaggi nel gioco, creati da una determinata scena.

3.3. Segnali

I segnali sono un modo per far comunicare i nodi in maniera asincrona in Godot. Ogni classe presenta dei segnali preimpostati ed emessi in determinati momenti, ad esempio quando un nodo viene caricato, questo emette il segnale *ready*, oppure quando un bottone viene premuto, viene emesso il segnale *pressed*.

I segnali inoltre possono anche contenere dei parametri, che possono essere utilizzati per passare informazioni tra i nodi.

Infine, si possono creare segnali personalizzati, che possono essere emessi in qualsiasi momento dal nodo che li ha definiti tramite il metodo *signal.emit(...)*.

Ci sono due modi per collegare un segnale ad un altro nodo:

- tramite l'editor: selezionando il nodo che emette il segnale e trascinandolo sul nodo che deve ricevere il segnale, e selezionando il metodo che deve essere chiamato quando il segnale viene emesso;
- tramite codice: utilizzando il metodo *signal.connect(Callable)* del nodo che emette il segnale, passando come parametro il nome del segnale e il nodo che deve ricevere il segnale, e il metodo che deve essere chiamato quando il segnale viene emesso.

4. Architettura

4.1. Introduzione

Di seguito viene presentata l'architettura del progetto tramite diagrammi UML delle classi, che mostrano le relazioni tra le classi utilizzate nel progetto.

L'applicazione è strutturata come un monolite, decisione presa per i seguenti motivi:

- **semplicità:** l'applicazione è relativamente piccola e non richiede una struttura complessa per essere gestita;
- **facilità di manutenzione:** la struttura monolitica permette di gestire più facilmente le dipendenze tra le classi, poiché tutte le classi sono contenute in un unico file e non è necessario gestire le dipendenze tra moduli diversi;
- **migliori prestazioni:** in un videogioco le prestazioni sono fondamentali e una struttura monolitica può contribuire a ottimizzare le performance, riducendo i tempi di caricamento e migliorando la fluidità del gioco.

4.1.1. Funzioni comuni

Molte classi del progetto presentano delle funzioni virtuali comuni, che vengono fornite dalle classi base del motore di gioco:

+ready(): void

Questa funzione viene chiamata quando il nodo è pronto per essere utilizzato, ovvero quando tutti i nodi figli sono stati caricati e il nodo è pronto per essere utilizzato.

In questa funzione è possibile inizializzare le variabili, collegare i segnali e impostare le proprietà del nodo.

È importante notare che questa funzione viene chiamata solo una volta, quando il nodo viene caricato per la prima volta nella scena, e non ad ogni frame del gioco.

+process(delta: float): void

Questa funzione viene chiamata ad ogni frame del gioco e permette di aggiornare lo stato della classe ad ogni frame. Il parametro *delta* rappresenta il tempo trascorso dall'ultimo frame, ed è utile per gestire le animazioni e le interazioni in modo fluido e coerente.

+physics_process(delta: float): void

Questa funzione viene chiamata ad ogni frame di fisica del gioco, che di default è fisso 60 volte al secondo, anche se il numero di frame al secondo del gioco sia inferiore.

Questa funzione è utile per gestire le interazioni fisiche tra gli oggetti, come ad esempio la gestione delle collisioni e la gestione della gravità.

Il parametro *delta* rappresenta il tempo trascorso dall'ultimo frame di fisica.

Funzione chiamata ogni volta che il gioco rileva un qualsiasi input, sia da tastiera che da joystick. Il parametro *event* rappresenta l'input che chiama la funzione.

Di seguito viene mostrata l'architettura degli [asset*](#) presenti in più parti del gioco che non sono unici ad un livello specifico.

```

classDiagram
    class Collectibles {
        count_red : int = 0
        count_blue : int = 0
        count_green : int = 0
        -ready(): void
        +emit_signals(): void
        +add(value: int, groups: Array[StringName]): void
        +get_save_values(): void
        +reset_count(): void
        -on_player_save_handler_td_changed(red: int, green: int, blue: int): void
    }
    class PlayerUI {
        +red_counter : PanelContainer
        +green_counter : PanelContainer
        +blue_counter : PanelContainer
        +show_collectibles_count(red: bool, green: bool, blue: bool): void
        -on_collectibles_changed_blue(count: int): void
        -on_collectibles_changed_green(count: int): void
        -on_collectibles_changed_red(count: int): void
    }
    class PlayerSavesHandler {
        +red_training_data_count : int
        +blue_training_data_count : int
        +green_training_data_count : int
        +set_save_node(): void
        +set_red_training_data(value: int): void
        +set_green_training_data(value: int): void
        +set_blue_training_data(value: int): void
        +func load_data(): void
    }
    class PlayerSaves {
        +save_data(): void
        +load_data(): void
    }
    class Movement {
        +player : Player
        -ready(): void
        +get_move_input(delta: float, weight: float): void
    }
    class Player {
        -camera : Camera3D
        -state_machine : StateMachine
        -camera_raycast : CameraRaycast
        -spring_arm : SpringArm3D
        -collectibles : PlayerCollectibles
        +model : Node3D
        -model_player : AnimationPlayer
        -respawn_point : RespawnPoint
        -movement_node : PlayerMovement
        -bone_attachment : BoneAttachment3D
        +particle_emitter : ParticleEmitter
        +grab_item : GrabItem
        -ui : Control
        +saves_handler : SavesHandler
        -physics_process(delta: float): void
        +check_if_on_floor(delta: float): void
        +get_move_input(delta: float, weight: float): void
        +add_collectibles(value: int, groups: Array[StringName]): void
        +rotate_model_direction(): void
        +set_respawn_point(): void
        +return_to_respawn_point(): void
        +get_spring_arm(): void
        +play(animation: String): void
        +queue(animation: String): void
        +is_moving(): bool
        +reset_camera(): void
        +change_state(state: String, msg : Dictionary): void
        +change_collectibles_max_value(value: int): void
        +func lock_camera(value: bool): void
        +face(object: Node3D): void
        +get_current_state_name(): String
        +show_collectibles_count(red: bool, green: bool, blue: bool): void
        +reset_collectibles_count(): void
        +reposition_camera(target = self): void
    }
    class CharacterBody3D {
        +velocity: Vector3
        +gravity: Vector3
        +move_and_slide(): void
    }
    class Timer {
        +wait_timer: float
    }
    class ParticleEmitter {
        +start_run_particles(value: bool): void
        +load_run_particles(): void
        +load_jump_particles(): void
    }
    class CameraRaycast {
        -camera_pivot : CameraTarget
        -camera : Camera3D
        -spring_arm : SpringArm3D
        -focus_target : CameraFocusTarget
        +is_locked : bool
        -ready(): void
        -physics_process(delta: float): void
        +calculate_position(): void
        +respawn(point: Vector3): void
        +reset(keep_rotation : bool = false): void
        +reposition_camera(target: Vector3): void
        +zoom_in(value: float): void
        +lerp_on_target(lerp_target: Vector3, zoom_value: float): void
        +set_focus_rotation(): void
    }
    class GrabItem {
        +grabbable_item : Node3D
        +can_grab : bool
        +hold_item : Node3D
        +is_holding : bool
        +set_grab_item(body: Node3D): void
        +set_hold_item(): void
        +carry(): void
        +release(): void
        +can_grab_item(): bool
        +can_release_item(): bool
        +exit_area(body: CharacterBody3D): void
    }
    class StateMachine {
        +initial_state: State
        +state: State
        +get_initial_state(): State-ready(): void
        +unhandled_input(event: InputEvent): void
        -process(delta: float): void
        -physics_process(delta: float): void
        -transition_to_next_state(target_state_path: String, data: Dictionary): void
        -transition_to(state: String, msg : Dictionary): void
    }
    class Signal_timeout {
        <<Signal>>
        timeout
    }
    Collectibles --> Player
    PlayerUI --> Player
    PlayerSavesHandler --> Player
    PlayerSaves --> PlayerSavesHandler
    Movement --> Player
    Player --|> CharacterBody3D
    Player --> Timer
    Player --> ParticleEmitter
    Player --> CameraRaycast
    Player --> GrabItem
    Player --> StateMachine
    Timer --> Signal_timeout
    Signal_timeout --> ParticleEmitter
    
```

Il giocatore può essere considerata la classe principale di tutta l'applicazione, attraverso il quale l'utente può interagire con la maggior parte dell'applicazione.

9

per implementare un singleton in Godot è richiesto che questo sia caricato come [autoload*](#) in ogni scena, e non c'è motivo di caricare il giocatore nel menu principale, all'avvio del gioco.

Molte variabili presenti nel giocatore sono riferimenti ai suoi nodi figli presenti nella scena, queste variabili sono precedute dalla parola chiave *@onready* nel codice. Similmente, molte funzioni della classe servono solo per accedere alle variabili dei suoi nodi figli.

La classe del giocatore ha associate le seguenti classi divise per funzionalità:

- **Movement**
- **CameraRaycast**
- **StateMachine**
- **PlayerSavesHandler**
- **GrabItem**
- **ParticleEmitter**
- **PlayerUI**

4.2.1.1. Movement

La classe «Movement» si occupa di gestire gli input per il movimento del giocatore, la funzione *get_move_input* si occupa di prendere l'input del movimento e ruotarlo in base alla rotazione della telecamera.

4.2.1.2. CameraRayCast

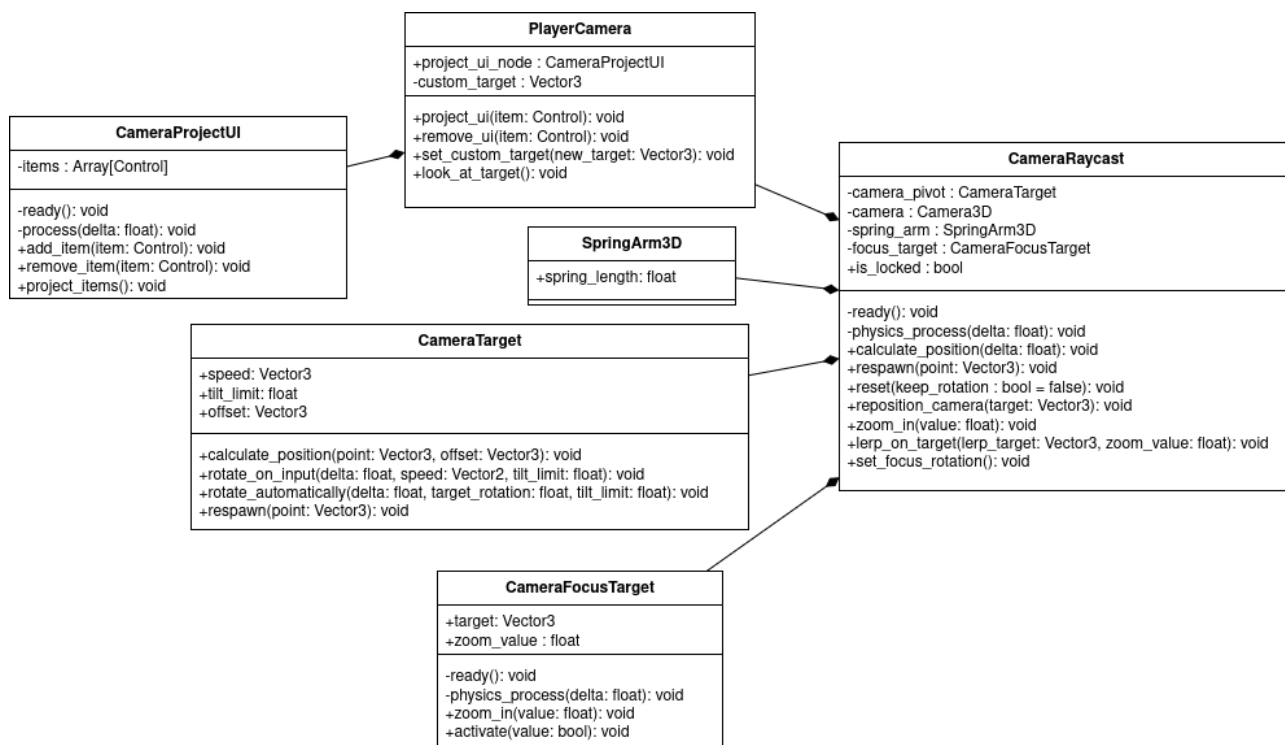


Figura 3: Diagramma delle classi della telecamera del giocatore

La telecamera del giocatore viene gestita da più classi per garantire diverse funzionalità tra quali la rotazione intorno al giocatore, proiezione di elementi della UI sullo schermo ed evitare che la telecamera passi attraverso i muri, generando il fenomeno chiamato [clipping*](#).

- **CameraRayCast**

Oltre che a gestire il lavoro di tutte le altre classi per il corretto funzionamento della telecamera, questa classe lancia un raggio dalla posizione del giocatore verso il basso per controllare velocemente la distanza dal terreno del giocatore. Nel caso il raggio tocchi ancora il terreno, la telecamera rimane per terra e non si sposta in alto con il giocatore.

- **PlayerCamera**

La telecamera effettiva, eredita dalla classe di Godot «Camera3D». Offre il metodo *look at target* che si occupa di girare la telecamera verso un obiettivo. Questo metodo viene usato da «CameraRayCast» per girare la telecamera verso un punto calcolato da quest'ultima.

- **CameraProjectUI**

La classe «CameraProjectUI» gestisce gli elementi della UI la cui posizione viene «proiettata» dallo spazio 3D del gioco, allo spazio 2D dello schermo. Contiene un array, composto da questi elementi. Nel caso l'array sia vuoto, la modalità di processo viene disabilitata, cioè le operazioni della classe non vengono più effettuate ad ogni fotogramma del gioco.

- **SpringArm3D**

Classe fornita da Godot. La sua posizione globale corrisponde sempre a quella del giocatore, e si occupa di avvicinare la telecamera quando è vicino ad un muro per evitare il clipping.

- **CameraTarget**

La classe «CameraTarget» eredita dalla classe di Godot «Marker3D». La sua posizione viene calcolata da «CameraRaycast» e si occupa di gestire gli input per ruotare e muovere la telecamera intorno al giocatore.

- **CameraFocusTarget**

La classe «CameraFocusTarget» si occupa di gestire lo spostamento della telecamera in casi speciali, ad esempio durante un dialogo con un personaggio non giocabile, dove la telecamera deve girarsi verso il personaggio che sta parlando.

4.2.1.3. StateMachine

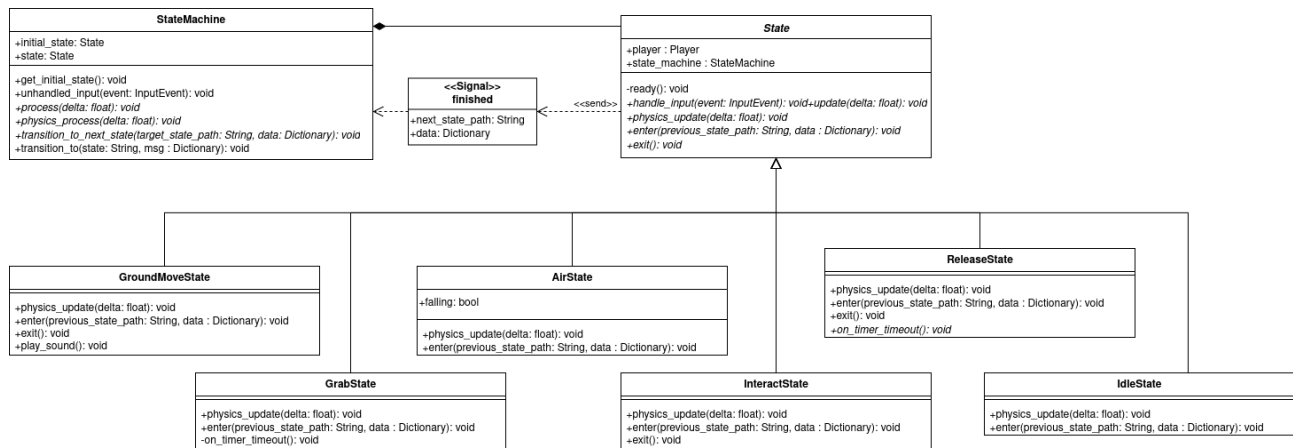


Figura 4: Diagramma sulla struttura della macchina di stati

La macchina di stati è stata utilizzata per controllare meglio i diversi stati in cui il personaggio del giocatore può eseguire, esempio: il movimento, salto... L'uso della macchina di stati, inoltre, ha garantito una gestione più semplice del personaggio del giocatore e ha reso più facile aggiungere funzionalità a questo. La figura sotto mostra il possibile flusso degli stati del giocatore.

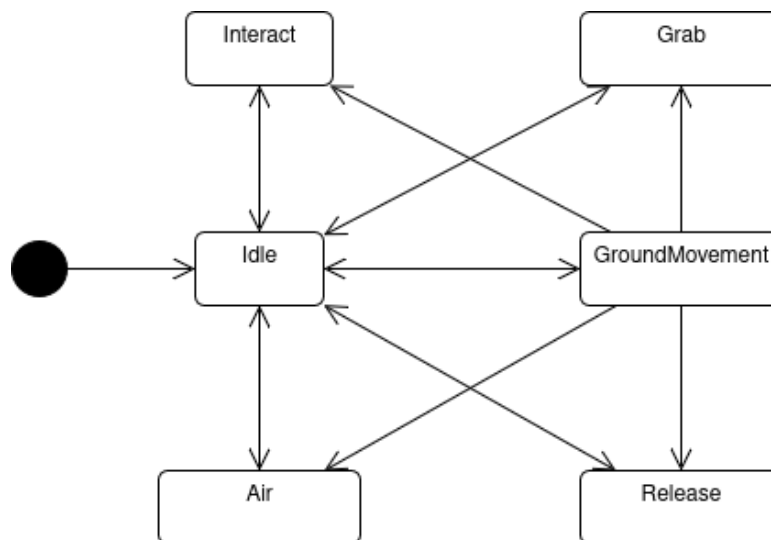


Figura 5: Diagramma degli stati

• StateMachine

La classe «StateMachine» si occupa di gestire la transizione degli stati. L'attributo *state* indica lo stato corrente del personaggio del giocatore. Quando riceve il segnale «finished» dallo stato

in cui si trova, si occupa di passare allo stato indicato dal segnale, passando gli eventuali dati contenuto nel Dictionary allo stato successivo.

- **State**

Classe base astratta per tutti gli stati. Include un riferimento al giocatore ed alla macchina di stati. Fornisce i seguenti metodi virtuali per le classi figlie:

- *+enter(previous_state_path: String, data: Dictionary)*

Chiamato non appena lo stato diventa attivo. L'attributo *data* contiene dei possibili dati mandati dallo stato precedente.

- *+exit()*

Chiamato quando si esce dallo stato

- *+physics_update(delta: float)*

Chiamato ad ogni frame fisico dello stato quando questo è **attivo**.

- *+update(delta: float)*

Chiamato ad ogni frame dello stato quando questo è **attivo**.

- *+handle_input(event: InputEvent)*

Chiamato quando viene premuto un input quando lo stato è attivo. L'attributo *event* rappresenta l'input premuto.

- **IdleState**

Lo stato «Idle» è lo stato iniziale del giocatore. Questo stato viene chiamato quando il giocatore è fermo per terra. Può passare a tutti gli altri stati in base agli input premuti in diverse condizioni. Ad esempio se il giocatore sta portando qualcosa e preme il tasto di interazione, il personaggio passa allo stato «Release», ma se non sta portando niente, allora non succede niente. Invece se preme lo stesso tasto dentro un'area specifica, il personaggio passa allo stato «Interact»

- **GroundMovementState**

Il personaggio del giocatore passa allo stato «GroundMovement» quando viene premuto un input per spostarsi rimanendo per terra. Come lo stato «Idle», si può passare a tutti gli altri stati anche da questo, seguendo le stesse condizioni dello stato «Idle».

- **AirState**

Si può passare a questo stato in due condizioni: il giocatore cade da una piattaforma, o preme il tasto per saltare. Nell'ultimo caso, lo stato precedente manda un valore «jump = true» all'interno del Dictionary, in questo modo lo stato controlla se è presente il medesimo valore ed in caso positivo, esegue il salto, caricando la rispettiva animazione e modificando la velocità verticale.

- **InteractState**

Lo stato «Interact» indica che il personaggio del giocatore è impegnato ad interagire con un'altra entità, ad esempio mentre parla con un personaggio non giocabile o legge un cartello. A differenza degli altri stati, non è un input a far cambiare stato, ma i segnali dalle entità esterne.

- **GrabState**

Il personaggio del giocatore passa allo stato «Grab» quando il giocatore preme il tasto per prendere un oggetto vicino a uno di questi. Importante notare che questo stato rappresenta solo quando il personaggio prende un oggetto, dopo aver svolto l'azione, il personaggio torna allo stato «Idle», cambiando le animazioni in modo che rispecchino la situazione.

- **ReleaseState**

Quando il giocatore preme di nuovo il tasto per prendere un oggetto mentre il personaggio sta portando un oggetto, questo passa allo stato «Release» e lascia l'oggetto. Come il suo stato opposto, una volta lasciato l'oggetto, il giocatore torna allo stato «Idle».

4.2.1.4. PlayerUI

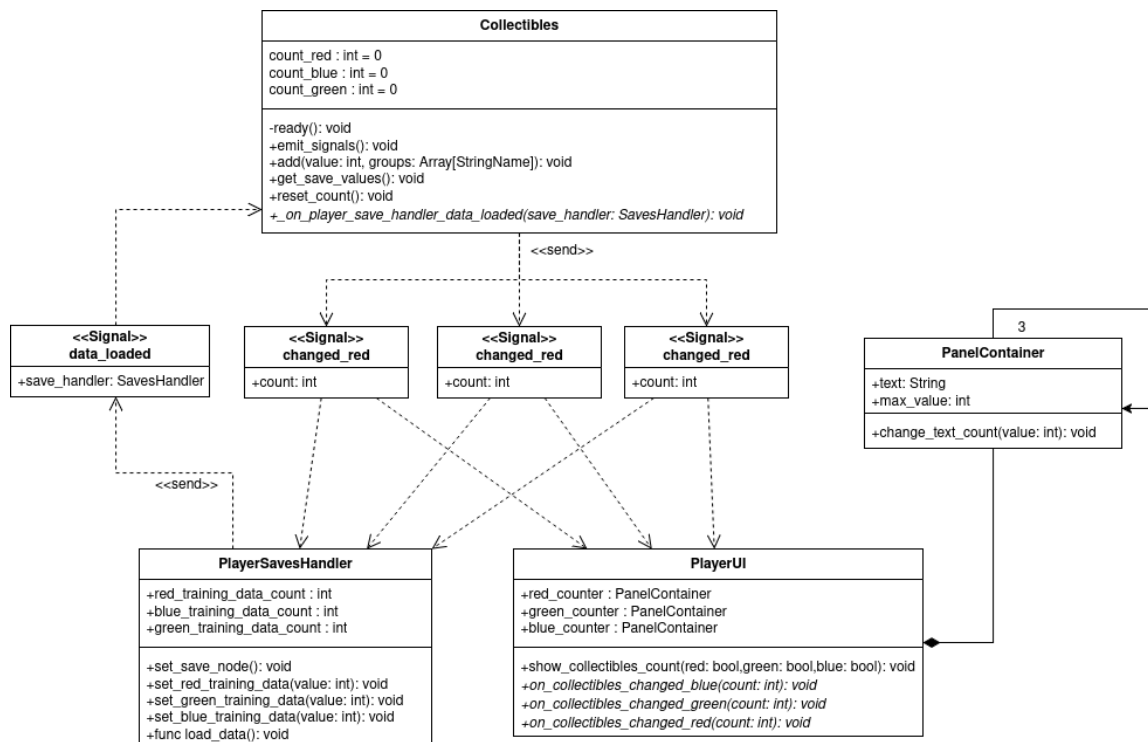


Figura 6: Diagramma delle classi della UI del giocatore

Il giocatore presenta la sua UI personale che viene caricata insieme a il giocatore. Nella UI viene visualizzato il numero di *training_data* raccolti, divisi per tipo.

- **Collectibles:**

La classe «Collectibles» contiene i dati effettivi sul numero dei *training_data* raccolti. Ogni volta che il valore viene aggiornato, manda il rispettivo segnale in modo da aggiornare il conteggio anche nella UI.

- **PlayerUI:**

La classe «PlayerUI» gestisce, appunto, la UI del giocatore. Riceve i segnali di «Collectibles» e aggiorna i valori. La classe è composta da 3 «PanelContainer», ognuno che contiene il numero del suo rispettivo tipo di *training_data*.

4.2.1.5. PlayerSavesHandler

La classe «PlayerSavesHandler» gestisce i salvataggi del giocatore. Vengono salvati tre attributi, il numero di ognuno dei *training_data* raccolti per i livelli. Ogni volta che il giocatore raccoglie un *training_data*, il valore viene aggiornato nella classe, e viene chiamato il metodo fornito dalla classe base *save_data()* per salvare i dati nel file «./player_save.ini».

4.2.1.6. GrabItem

La classe «GrabItem» si occupa di controllare quando il giocatore si avvicina ad un oggetto che può prendere. Il controllo viene fatto da un'«Area3D», quando un oggetto che può essere preso entra in quest'area, viene avisato il giocatore che può afferrare l'oggetto.

4.2.1.7. ParticleEmitter

La classe «ParticleEmitter» si occupa di caricare le «GPUParticles3D», cioè le particelle o effetti, in specifiche condizioni. Ad esempio quando il giocatore salta, carica le particelle del salto, quando corre, carica il «fumo» della corsa.

4.2.2. Interazione

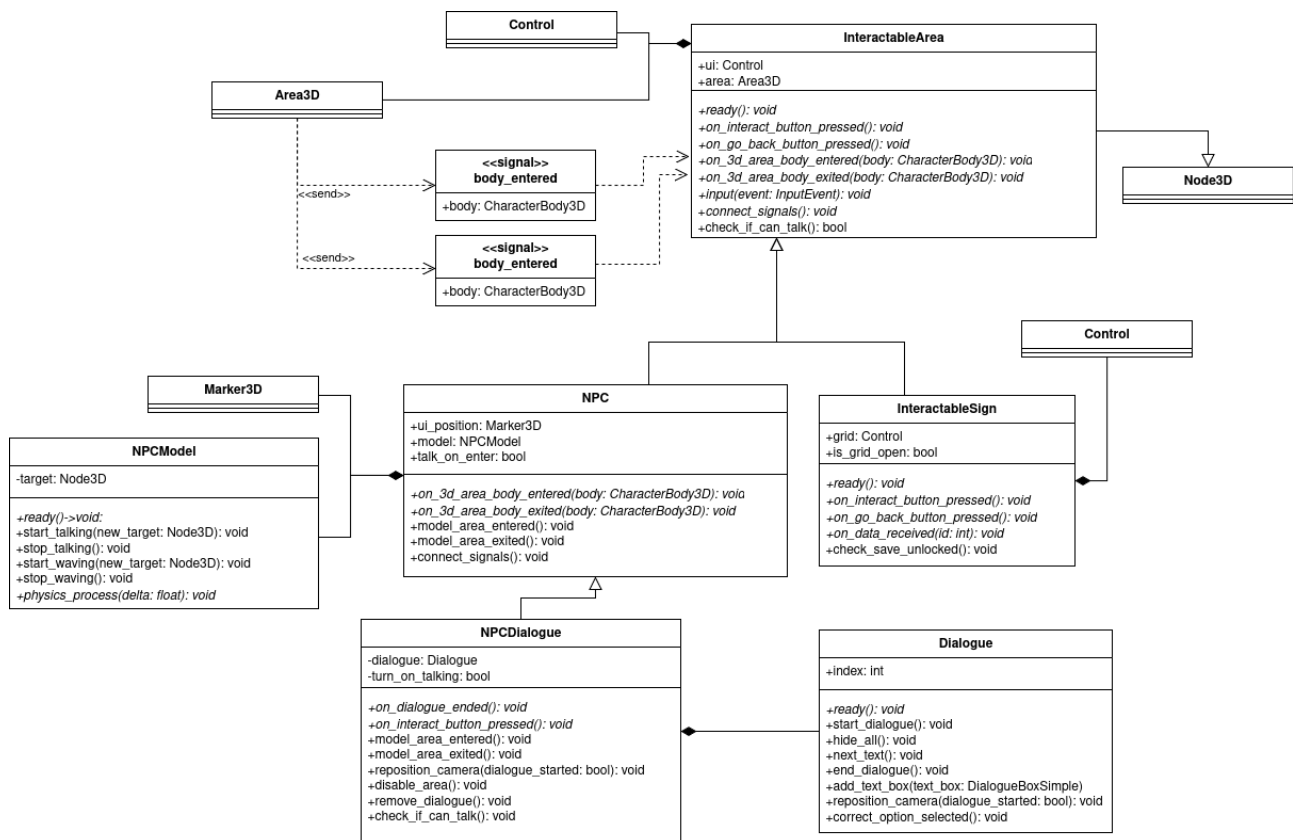


Figura 7: Diagramma degli oggetti con cui il giocatore può interagire

Nei livelli sono presenti diverse entità con cui il giocatore può interagire. Di seguito vengono descritte i diversi tipi di entità, e le classi che le compongono.

4.2.2.1. InteractableArea

Classe base astratta che fornisce i metodi alle classi figlie. La classe è composta da una classe Area3D, che invia i segnali quando il giocatore entra ed esce, e da una classe «Control» che rappresenta la UI che il giocatore visualizza quando entra. La classe è composta da una classe Area3D, che invia i segnali quando il giocatore entra ed esce, e da una classe «Control» che rappresenta la UI che il giocatore visualizza appena entra nell'area.

4.2.2.2. Control

Di seguito vengono mostrati i vari tipi di UI che il giocatore può visualizzare quando entra nell'area.

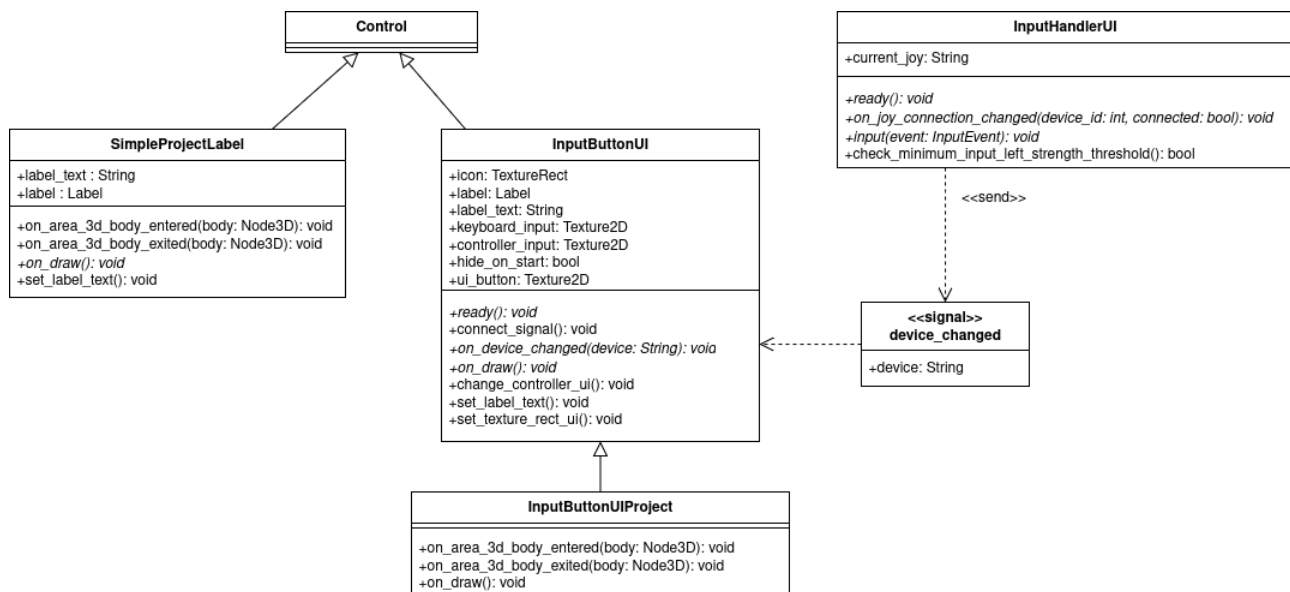


Figura 8: Diagramma dei vari tipi di UI

• SimpleProjectLabel

Viene usata dalla classe «NPC» come messaggio semplice e rappresenta un elemento UI che viene proiettato nello spazio 2D della telecamera. Appena il giocatore entra nell'area, la classe manda sé stessa come riferimento alla classe «CameraProjectUI». Questa la aggiunge all'array, attivando la classe. Quando il giocatore esce, l'elemento viene tolto dall'array.

• InputButtonUI

La classe «InputButtonUI» rappresenta un messaggio che contiene l'input da premere. L'immagine dell'input da premere cambia in base al dispositivo connesso: viene mostrato il tasto della tastiera se il giocatore sta utilizzando la tastiera, il tasto del joystick se sta utilizzando un joystick. Il cambio dell'immagine dell'input viene chiamato dal segnale *device_changed*, mandato dal singleton «InputHandlerUI», con attributo il nome del controller collegato. Questa classe viene usata da oggetti inanimati come un cartello.

• InputButtonUIProject

La classe «InputButtonUIProject» contiene sempre l'immagine dell'input da premere, l'unica differenza è che questo elemento viene proiettato nello spazio 2D della telecamera. Viene usata dalla classe «NPCDialogue».

4.2.2.3. NPC

La classe «NPC» rappresenta un personaggio non giocabile che ha assegnato una semplice frase come messaggio. Questa frase viene visualizzata in una classe «SimpleProjectLabel» il cui funzionamento è stato spiegato nella sezione precedente. Presenta anche una classe

4.2.3.1. Dialogue

text

4.2.3.2. DialogueBoxSimple

text

4.2.3.3. DialogueBoxOptions

text

4.2.3.4. DialogueOptionsButtons

text

4.2.4. Salvataggi

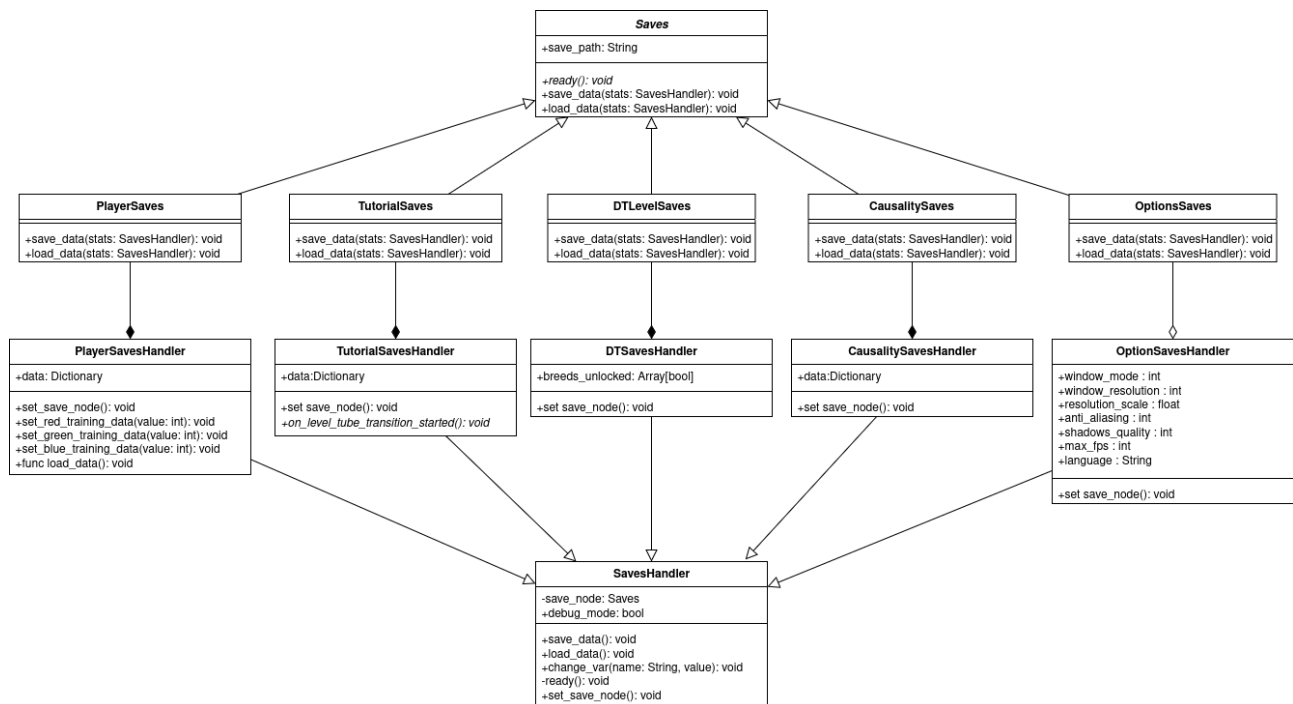


Figura 10: Diagramma sul funzionamento dei salvataggi

4.2.4.1. Saves

4.2.4.2. SavesHandler

4.2.5. Singletons / Autoloads

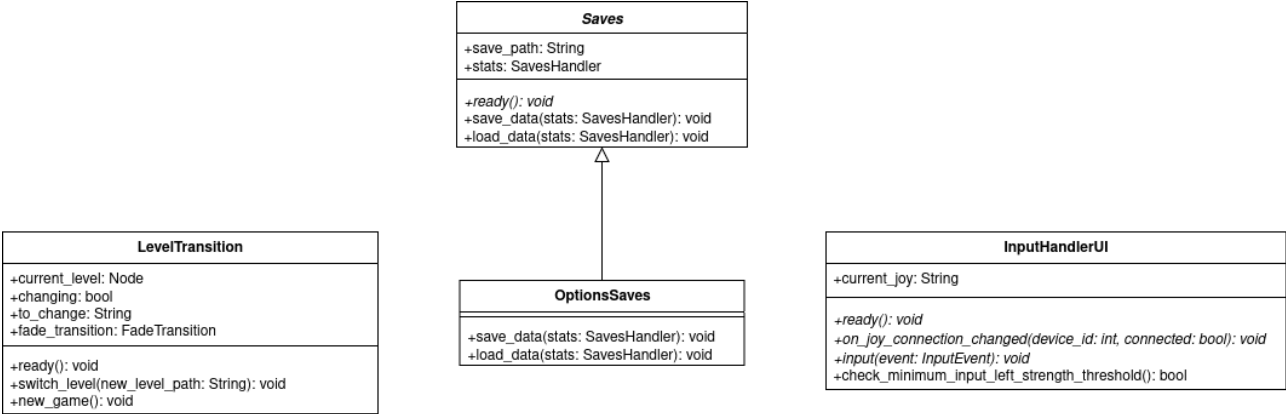


Figura 11: Classi *Autoloads*

4.2.5.1. LevelsTransition

text

4.2.5.2. OptionsSave

text

4.2.5.3. InputUIHandler

text

4.3. Struttura base livello

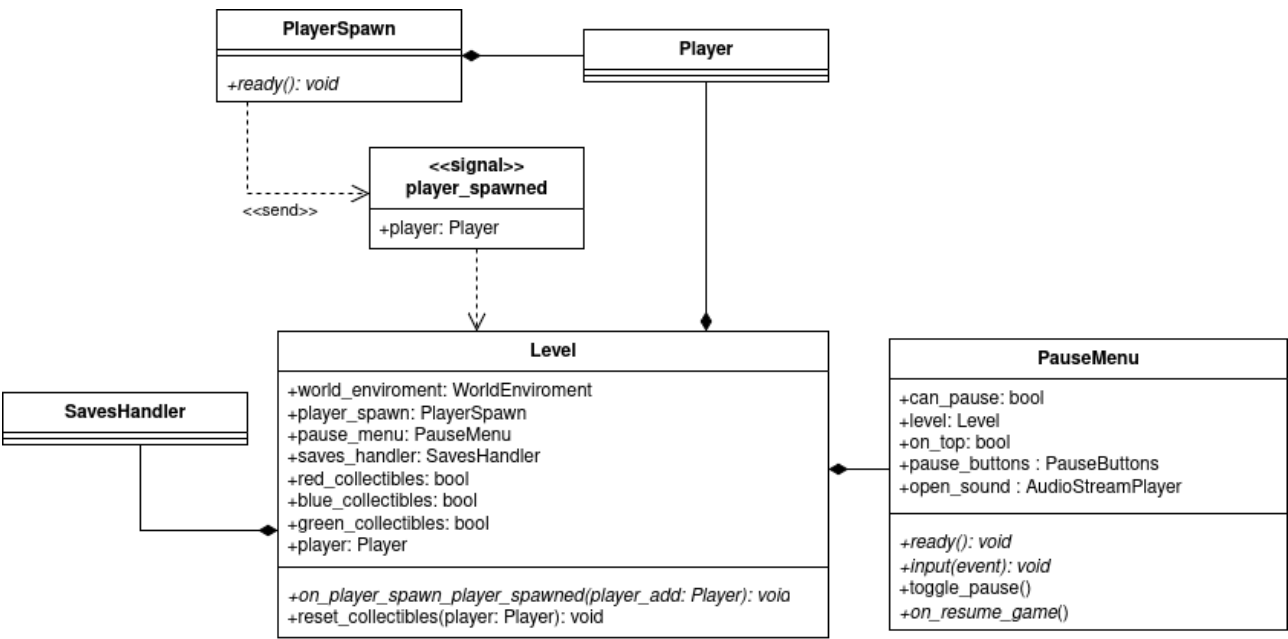


Figura 12: Diagramma di un livello base

text

4.3.1. PlayerSpawn

text

4.3.2. PauseMenu

text

4.4. Livello «Regressione lineare»

4.4.1. Cannone e grafico LR

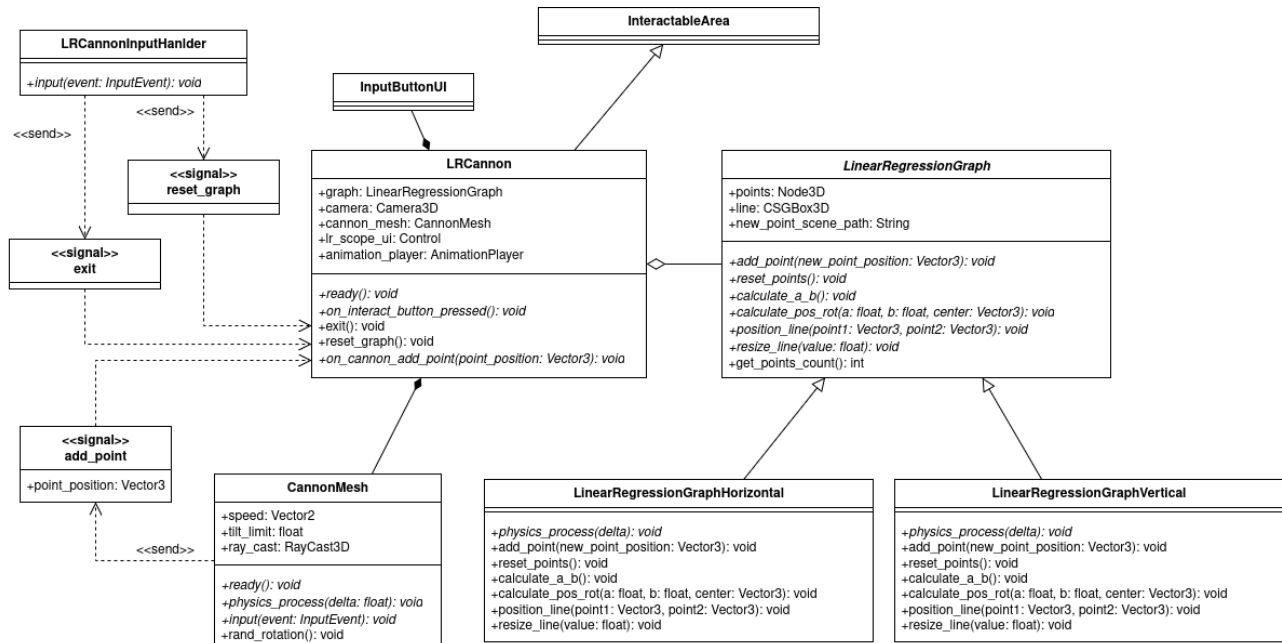


Figura 13: Diagramma sul funzionamento di un grafico «Linear Regression» nel gioco

4.4.1.1. LRCannon

4.4.1.2. LinearRegressionGraph

4.5. Livello «Albero di decisione»

4.5.1. Albero

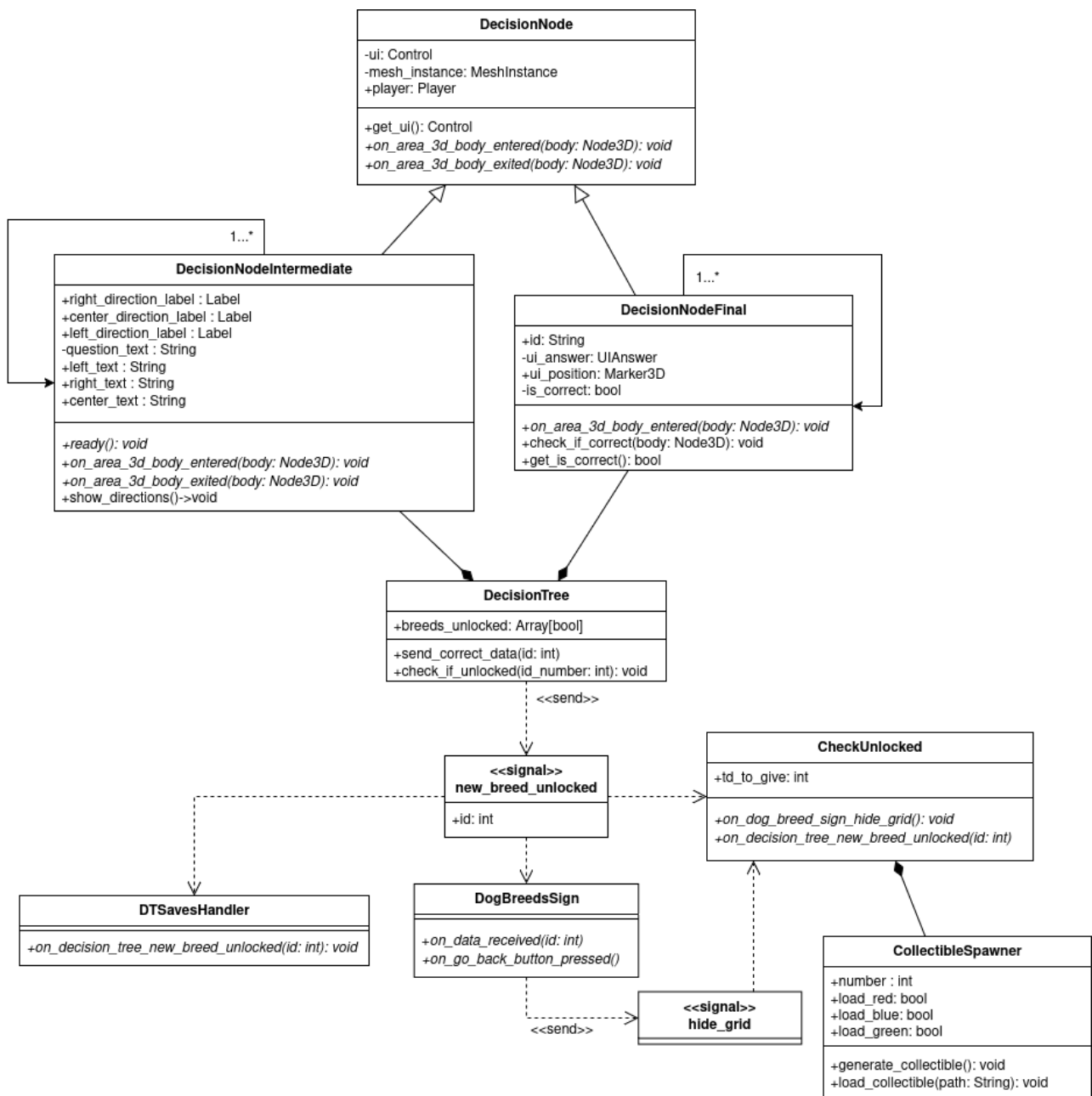


Figura 14: Diagramma sul funzionamento dell'Albero di decisione

text

4.5.1.1. DecisionNode

4.5.1.2. DecisionTree

4.5.1.3. CheckUnlocked

4.5.1.4. CollectibleSpawner

4.6. Livello «Causalità»

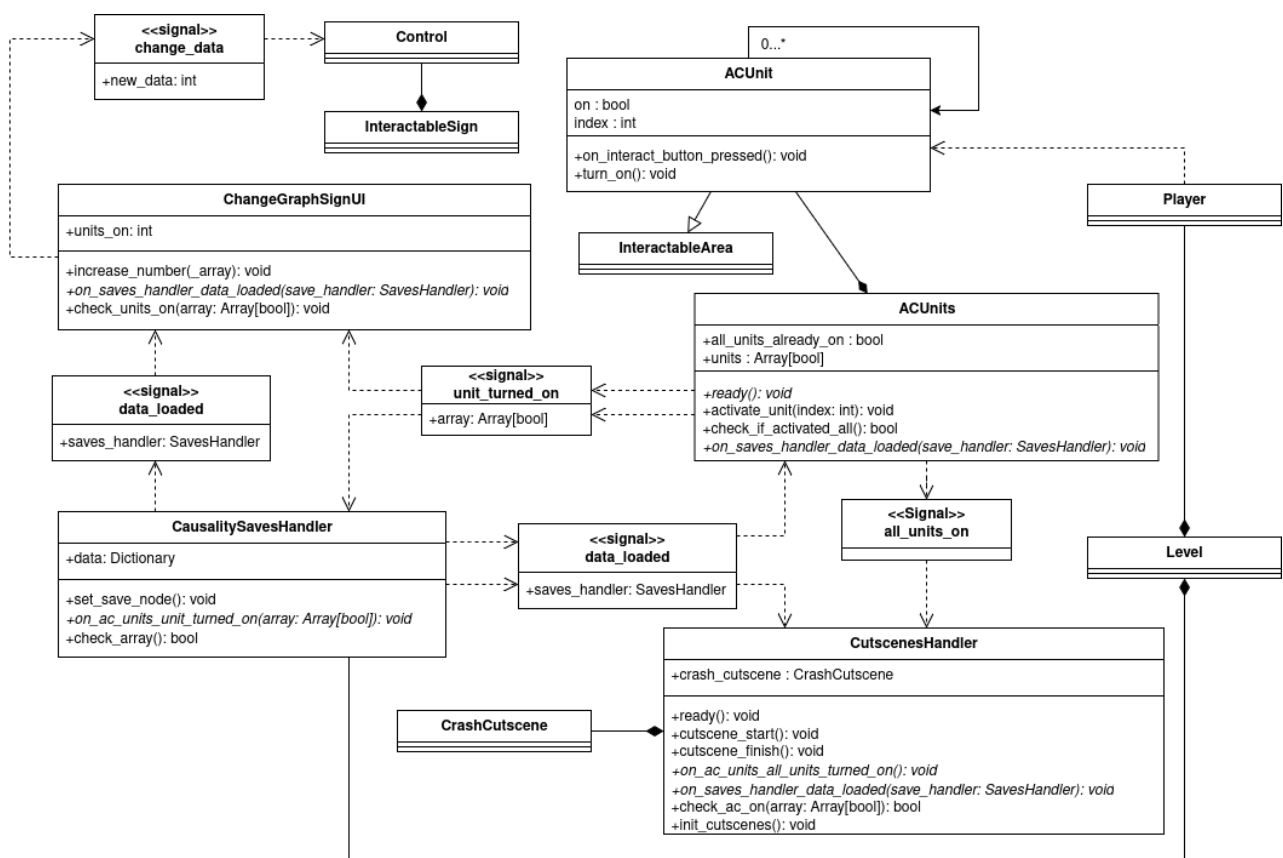


Figura 15: Diagramma del livello della causalità

4.6.1. CausalitySavesHandler

4.6.2. ACUnit

4.6.3. ACUnits

4.6.4. CutsscenesHandler

4.6.5. Scena di intermezzo

5. Requisiti soddisfatti

Nella seguente sezione vengono presentati tutti i requisiti presenti nel documento *Analisi dei requisiti*, presentando il loro stato di soddisfazione.

5.1. Tabella requisiti soddisfatti

ID Requisito	Descrizione	Soddisfatto
R-01-F-O	Il giocatore deve essere in grado di muoversi in uno spazio tridimensionale	✓
R-02-F-O	Il giocatore deve essere in grado di muoversi in uno spazio tridimensionale portando un oggetto	✓
R-03-F-O	Il giocatore deve essere in grado saltare	✓
R-04-F-O	Il giocatore deve essere in grado di saltare con un oggetto in mano	✓
R-05-F-O	La telecamera deve muoversi automaticamente dietro il giocatore quando questo si muove	✓
R-06-F-O	Il giocatore deve essere in grado di ruotare la telecamera	✓
R-07-F-O	Il giocatore deve tornare in una posizione precedente quando cade dal livello	✓
R-08-F-O	Il giocatore deve essere in grado di raccogliere collezionabili sparsi per il livello	✓
R-09-F-O	Il giocatore deve essere in grado di interagire con entità presenti nel livello	✓
R-10-F-O	Il giocatore deve essere in grado di visualizzare subito il messaggio di un'entità automatica	✓
R-11-F-O	Il giocatore deve essere in grado di parlare con un personaggio non giocabile	✓
R-12-F-O	Il giocatore deve poter visualizzare il dialogo quando interagisce con delle entità specifiche	✓
R-13-F-O	Il giocatore deve avere la possibilità di prendere una decisione quando gli viene mostrato nel dialogo	✓

ID Requisito	Descrizione	Soddisfatto
R-14-F-O	Il giocatore deve poter andare avanti nel dialogo	✓
R-15-F-O	Il giocatore deve poter finire l'interazione con un'entità	✓
R-16-F-O	Il giocatore deve poter essere in grado di raccogliere oggetti	✓
R-17-F-O	Il giocatore deve poter essere in grado di lasciare l'oggetto che sta portando	✓
R-18-F-O	Il giocatore deve poter interagire con un cartello in un livello	✓
R-19-F-O	Il giocatore deve poter visualizzare il contenuto di un cartello	✓
R-20-F-O	Il giocatore deve poter visualizzare gli oggetti che ha classificato correttamente	✓
R-21-F-O	Il giocatore deve poter visualizzare il grafico di un cartello	✓
R-22-F-O	Il giocatore deve poter modificare i dati presenti in un cartello	✓
R-23-F-O	Il giocatore deve poter essere in grado di interagire con un'area di transizione	✓
R-24-F-O	Il giocatore deve poter essere in grado di cambiare livello	✓
R-25-F-O	Il giocatore deve poter interagire con la macchina LR	✓
R-26-F-O	Il giocatore deve poter inserire punti nel grafico LR	✓
R-27-F-O	Il giocatore deve poter resettare i punti aggiunti nel grafico LR	✓
R-28-F-O	Il giocatore deve poter salire sopra un nodo dell'albero di decisione	✓

ID Requisito	Descrizione	Soddisfatto
R-29-F-O	Il giocatore deve poter visualizzare le scelte da prendere sopra il nodo	✓
R-30-F-O	Il giocatore deve poter piazzare un oggetto sopra un nodo dell'albero di decisione	✓
R-31-F-O	Il giocatore deve poter visualizzare se l'oggetto posto sul nodo sia giusto	✓
R-32-F-O	Il giocatore deve poter visualizzare se l'oggetto posto sul nodo sia sbagliato	✓
R-33-F-O	Il giocatore deve poter salvare il gioco in momenti specifici	✓
R-34-F-O	Il giocatore deve poter mettere in pausa il gioco	✓
R-35-F-O	Il giocatore deve poter riprendere il gioco dal menu di pausa	✓
R-36-F-O	Il giocatore deve poter accedere alle opzioni del gioco	✓
R-37-F-O	Il giocatore deve poter tornare al livello hub dal menu di pausa	✓
R-38-F-D	Il giocatore deve poter tornare al menu principale dal menu di pausa	✓
R-39-F-O	Il giocatore deve poter chiudere il gioco dal menu di pausa o principale	✓
R-40-F-O	Il giocatore deve poter caricare una partita salvata dal menu principale	✓
R-41-F-O	Il giocatore deve poter avviare una nuova partita dal menu principale	✓
R-42-F-O	Il giocatore deve poter modificare la modalità della finestra dal menu delle opzioni	✓
R-43-F-O	Il giocatore deve poter modificare la risoluzione della finestra	✓

ID Requisito	Descrizione	Soddisfatto
R-44-F-D	Il giocatore deve poter modificare la scala di risoluzione del gioco	✓
R-45-F-D	Il giocatore deve essere in grado di poter modificare il tipo di anti-aliasing usato nel gioco, oppure non usarlo	✓
R-46-F-D	Il giocatore deve essere in grado di modificare la qualità delle ombre nel gioco	✓
R-47-F-D	Il giocatore deve poter cambiare lingua di gioco	✓
R-48-F-D	Il giocatore deve poter cambiare il volume generale del gioco	✓
R-49-F-O	Il gioco deve applicare e salvare le opzioni selezionate	✓
R-50-F-O	Il giocatore deve poter accendere delle unità esterne di un condizionatore premendo un tasto	✓
R-51-F-O	Il giocatore deve poter vedere scene di intermezzo	✓
R-01-Q-O	È richiesta la presentazione del documento Specifica Tecnica che include dettagli riguardanti la progettazione architettuale	✓
R-02-Q-O	È richiesta la presentazione del documento Specifica Tecnica che include dettagli riguardanti le tecnologie utilizzate	✓
R-03-Q-O	È richiesta la presentazione del documento Specifica Tecnica che include dettagli riguardanti la progettazione della base di dati	✓
R-04-Q-O	È richiesta la presentazione del documento Specifica Tecnica che include dettagli riguardanti l'implementazione del sistema di raccomandazione utilizzato con LLM	✓
R-05-Q-O	Tutte le attività del progetto devono essere svolte rispettando le Norme di Progetto	✓

ID Requisito	Descrizione	Soddisfatto
R-06-Q-O	Tutto il codice e la documentazione vanno salvati all'interno di un repository pubblico	✓
R-01-V-O	Il gioco deve avere almeno 3 livelli	✓
R-02-V-O	Un livello deve avere come tema «Regressione lineare»	✓
R-03-V-O	Un livello deve avere come tema «Alberi di decisione»	✓
R-04-V-O	Un livello deve avere come tema «Causalità»	✓
R-05-V-O	Deve essere presente un livello «Tutorial» che insegni al giocatore i comandi base	✓
R-06-V-O	Il movimento del gioco deve essere tridimensionale	✓
R-01-A-O	Il gioco deve supportare il sistema operativo Windows	✓
R-02-A-D	Il gioco deve supportare il sistema operativo Ubuntu	✓
R-03-A-D	Il gioco deve supportare il sistema operativo MacOS	
R-04-A-D	La piattaforma deve essere responsive e funzionare correttamente su dispositivi desktop con risoluzione minima di 640x360px	
R-05-A-O	Il gioco deve supportare input da tastiera	✓
R-06-A-D	Il gioco deve supportare input da un joypad generico	✓
R-07-A-O	Il gioco deve mostrare gli input del dispositivo che si sta usando	✓

Tabella 2: Requisiti di funzionalità

5.2. Grafico requisiti soddisfatti